System Design & Architecture Document

Project: Fair-Play Ticket Auction & Exchange Platform

---

1. High-Level Architecture

The system will follow a standard client-server architecture with a real-time event-driven component.

- **Client:** React.js (Handles UI, WebSocket connections for live bidding).
- **API Gateway/Backend:** Node.js with Express.js (REST APIs for CRUD operations, Socket.io for real-time bid broadcasting).
- **Database:** MongoDB (using multi-document ACID transactions for financial/ticket transfers).
- **Cache/Queue (Optional but recommended):** Redis (for managing rapid concurrent bids in memory before batch-writing to MongoDB).

---

2. Design Pattern Justifications & SOLID Principles

To build a professional-grade system, we must decouple components. Here is exactly why specific Gang of Four (GoF) patterns are chosen for this system:

A. Strategy Pattern (Behavioral)

Where it's used: The Auction Pricing Engine. **Why it's chosen:** We have different auction types (English Ascending, Vickrey Sealed-Bid). If we hardcode this logic into the `AuctionController`, it violates the **Open-Closed Principle (OCP).** By using the Strategy Pattern, we define an `AuctionStrategy` interface.

- In a Vickrey auction, truth-telling emerges as a weakly dominant strategy, meaning users are incentivized to bid their true maximum valuation. The algorithm isolates this logic: it finds the highest bidder but charges the second-highest bid.
- If we later want to add a Dutch Auction (descending price), we simply create a new `DutchAuctionStrategy` class without touching the core bidding engine.

B. Observer Pattern (Behavioral)

Where it's used: Real-Time Bid Notifications & WebSocket Broadcasting. **Why it's chosen:** When a bid is placed, multiple things need to happen: the current highest bidder needs an "Outbid" alert, the seller needs a "New Bid" alert, and the UI needs to update the current price.

Using the Observer pattern, the `Auction` object acts as the Subject. Whenever its state changes, it notifies all attached Observers (the WebSocket server, the email notification service) without needing to know their specific implementation details.

C. Mediator Pattern (Behavioral)

Where it's used: The Checkout & Escrow Settlement flow. **Why it's chosen:** When an auction closes, the system must: 1) Deduct funds from the buyer, 2) Transfer the ticket ownership, 3) Release escrowed funds back to losing bidders, 4) Pay the seller. If the `Buyer` object talks directly to the `Ticket` object and the `Seller` object, the code becomes tightly coupled spaghetti. The Mediator (an `AuctionSettlementService`) sits in the middle. The components only communicate with the Mediator, centralizing the complex transaction logic and ensuring database ACID compliance if one step fails.

---

3. Database Schema Design (MongoDB)

To handle concurrency and relational integrity in a NoSQL database, we will use a normalized approach with references, leveraging MongoDB's multi-document transactions for the settlement phase.

- **Users Collection:** `_id`, `name`, `email`, `walletBalance`, `isVerified`
- **Tickets Collection:** `_id`, `sellerId`, `eventId`, `seatInfo`, `status` (Available, In_Auction, Sold)
- **Auctions Collection:** `_id`, `ticketId`, `strategyType` (English, Vickrey), `startingPrice`, `endTime`, `currentHighestBid`, `winningBidderId`, `status` (Active, Closed)
- **Bids Collection:** `_id`, `auctionId`, `bidderId`, `amount`, `timestamp` *(Indexed by `auctionId` and `amount` for fast querying of the top bids).*

4. UML Class Diagram (Core Auction Engine)

Here is the structural representation of how the Strategy pattern is applied to the auction engine.

```
classDiagram
  class AuctionContext {
    - strategy: AuctionStrategy
    - auctionData: Auction
    + setStrategy(strategy: AuctionStrategy)
    + processBid(bid: Bid)
    + calculateClearingPrice() float
  }
```

```
class AuctionStrategy {
    <<interface>>
    + validateBid(currentHighest: float, newBid: float) boolean
    + determineWinnerAndPrice(bids: List~Bid~) SettlementDetails
}

class EnglishAuctionStrategy {
    + validateBid(currentHighest: float, newBid: float) boolean
    + determineWinnerAndPrice(bids: List~Bid~) SettlementDetails
}

class VickreyAuctionStrategy {
    + validateBid(currentHighest: float, newBid: float) boolean
    + determineWinnerAndPrice(bids: List~Bid~) SettlementDetails
}

AuctionContext o-- AuctionStrategy
AuctionStrategy <|.. EnglishAuctionStrategy
AuctionStrategy <|.. VickreyAuctionStrategy
```

5. Sequence Diagram: Real-Time Bid Flow

This diagram illustrates the flow of data when a user places a bid during an active English auction, showcasing the Observer pattern via WebSockets.

```
sequenceDiagram
    actor Bidder
    participant UI as React Client
    participant API as Express Server
    participant DB as MongoDB
    participant WS as Socket.io Server

    Bidder->>UI: Clicks "Place Bid ($50)"
    UI->>API: POST /api/bids {auctionId, amount}
    API->>DB: Start Transaction
    DB-->>API: Read current highest bid

    alt Bid is valid (Higher than current)
        API->>DB: Insert into Bids collection
        API->>DB: Update Auctions currentHighestBid
        API->>DB: Commit Transaction
        API-->>UI: 201 Created (Success)
        API->>WS: Emit 'NEW_BID' event
```

```
    WS-->>UI: Broadcast new price to all clients
  else Bid is invalid
    API->>DB: Abort Transaction
    API-->>UI: 400 Bad Request (Bid too low)
  end
```

6. Technical Justifications & Architecture Decisions

This section outlines the rationale behind the chosen technology stack and software design patterns, ensuring the system is scalable, maintainable, and adheres to SOLID principles.

6.1 Technology Stack Justifications

- **Why MongoDB (NoSQL) instead of a Relational SQL Database?**
  - **Schema Flexibility:** Event tickets are highly polymorphic. A tech conference ticket might contain workshop schedules and dietary tiers, while a stadium concert ticket requires strict section, row, and seat coordinates. MongoDB's document model handles this structural variance naturally without requiring sparse tables or complex `JOIN` operations.
  - **Concurrency & ACID Compliance:** Historically, SQL is preferred for financial transactions. However, by utilizing MongoDB's multi-document ACID transactions (introduced in v4.0+), we can guarantee that the escrow deduction and ticket ownership transfer happen atomically. We employ optimistic concurrency control to prevent race conditions when multiple users bid on the same document simultaneously.
- **Why Node.js & Express?**
  - **High Concurrency & Asynchronous I/O:** A live auction platform is inherently I/O-bound, not CPU-bound. In the final ten seconds of an auction, the server must handle thousands of rapid-fire bid requests and WebSocket emissions. Node.js's single-threaded, non-blocking event loop is uniquely suited to handle this massive volume of concurrent connections without the overhead of context-switching found in thread-per-request models.
- **Why React?**
  - **Real-Time State Synchronization:** As the WebSocket server pushes new highest bids every few milliseconds, the frontend UI must re-render specific components (like the price tag and the countdown timer) instantaneously. React's Virtual DOM allows for these rapid, granular updates without causing performance bottlenecks or requiring full-page reloads.

6.2 Design Pattern Justifications

- **Why the Strategy Pattern for the Auction Engine?**
  - **The Problem:** Hardcoding the logic for an English (ascending) auction and a Vickrey (sealed-bid, second-price) auction into a single `AuctionService`

creates a monolithic class that violates the Single Responsibility Principle and the Open-Closed Principle.
    - **The Solution:** The Strategy Pattern encapsulates each pricing algorithm into its own class. From a game theory perspective, these algorithms enforce entirely different economic behaviors—for instance, the Vickrey strategy mathematically incentivizes truth-telling (bidding your true maximum valuation) as a dominant strategy. If the platform later decides to add a Dutch Auction (descending price), the system can easily scale by adding a new `DutchAuctionStrategy` class without modifying the core transaction engine.
- **Why the Mediator Pattern for Checkout & Settlement?**
    - **The Problem:** When an auction closes, the system must interact with the `Buyer` (deduct funds), the `Seller` (add funds), the `Ticket` (change ownership), and the `NotificationService` (send receipts). If these entities call each other directly, they become tightly coupled, making the system brittle and impossible to unit test in isolation.
    - **The Solution:** The Mediator Pattern introduces an `AuctionSettlementService`. The buyer, seller, and ticket objects do not know about each other; they only communicate with the Mediator. This centralizes the complex, multi-step transaction workflow, ensuring that if one step fails (e.g., the payment gateway times out), the Mediator can safely trigger a database rollback for the entire operation.
- **Why the Observer Pattern for Live Bidding?**
    - **The Problem:** When a new highest bid is accepted, multiple disconnected parts of the system need to react immediately. The UI needs the new price, the previous high bidder needs an "outbid" alert, and the analytics engine needs to log the event.
    - **The Solution:** The Observer Pattern establishes a one-to-many dependency. The core `Auction` acts as the Subject. Whenever a valid bid alters its state, it blindly broadcasts a `StateChanged` event. Various Observers (like the Socket.io broadcaster and the Email Service) listen for this event and execute their specific logic. This decouples the core bidding math from the notification and networking layers.